arXiv:1706.02413v1 [cs.CV] 7 Jun 2017

PointNet++: Deep Hierarchical Feature Learning on Point Sets in a Metric Space

Charles R. Qi **Li Yi** **Hao Su** **Leonidas J. Guibas**
Stanford University

Abstract

Few prior works study deep learning on point sets. PointNet [20] is a pioneer in this direction. However, by design PointNet does not capture local structures induced by the metric space points live in, limiting its ability to recognize fine-grained patterns and generalizability to complex scenes. In this work, we introduce a hierarchical neural network that applies PointNet recursively on a nested partitioning of the input point set. By exploiting metric space distances, our network is able to learn local features with increasing contextual scales. With further observation that point sets are usually sampled with varying densities, which results in greatly decreased performance for networks trained on uniform densities, we propose novel set learning layers to adaptively combine features from multiple scales. Experiments show that our network called PointNet++ is able to learn deep point set features efficiently and robustly. In particular, results significantly better than state-of-the-art have been obtained on challenging benchmarks of 3D point clouds.

1 Introduction

We are interested in analyzing geometric point sets which are collections of points in a Euclidean space. A particularly important type of geometric point set is point cloud captured by 3D scanners, e.g., from appropriately equipped autonomous vehicles. As a set, such data has to be invariant to permutations of its members. In addition, the distance metric defines local neighborhoods that may exhibit different properties. For example, the density and other attributes of points may not be uniform across different locations — in 3D scanning the density variability can come from perspective effects, radial density variations, motion, etc.

Few prior works study deep learning on point sets. PointNet [20] is a pioneering effort that directly processes point sets. The basic idea of PointNet is to learn a spatial encoding of each point and then aggregate all individual point features to a global point cloud signature. By its design, PointNet does not capture local structure induced by the metric. However, exploiting local structure has proven to be important for the success of convolutional architectures. A CNN takes data defined on regular grids as the input and is able to progressively capture features at increasingly larger scales along a multi-resolution hierarchy. At lower levels neurons have smaller receptive fields whereas at higher levels they have larger receptive fields. The ability to abstract local patterns along the hierarchy allows better generalizability to unseen cases.

We introduce a hierarchical neural network, named as PointNet++, to process a set of points sampled in a metric space in a hierarchical fashion. The general idea of PointNet++ is simple. We first partition the set of points into overlapping local regions by the distance metric of the underlying space. Similar to CNNs, we extract local features capturing fine geometric structures from small neighborhoods; such local features are further grouped into larger units and processed to produce higher level features. This process is repeated until we obtain the features of the whole point set.

The design of PointNet++ has to address two issues: how to generate the partitioning of the point set, and how to abstract sets of points or local features through a local feature learner. The two issues

are correlated because the partitioning of the point set has to produce common structures across partitions, so that weights of local feature learners can be shared, as in the convolutional setting. We choose our local feature learner to be PointNet. As demonstrated in that work, PointNet is an effective architecture to process an unordered set of points for semantic feature extraction. In addition, this architecture is robust to input data corruption. As a basic building block, PointNet abstracts sets of local points or features into higher level representations. In this view, PointNet++ applies PointNet recursively on a nested partitioning of the input set.

One issue that still remains is how to generate overlapping partitioning of a point set. Each partition is defined as a neighborhood ball in the underlying Euclidean space, whose parameters include centroid location and scale. To evenly cover the whole set, the centroids are selected among input point set by a farthest point sampling (FPS) algorithm. Compared with volumetric CNNs that scan the space with fixed strides, our local receptive fields are dependent on both the input data and the metric, and thus more efficient and effective.

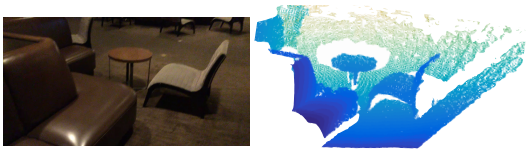


Figure 1: Visualization of a scan captured from a Structure Sensor (left: RGB; right: point cloud).

Deciding the appropriate scale of local neighborhood balls, however, is a more challenging yet intriguing problem, due to the entanglement of feature scale and non-uniformity of input point set. We assume that the input point set may have variable density at different areas, which is quite common in real data such as Structure Sensor scanning [18] (see Fig. 1). Our input point set is thus very different from CNN inputs which can be viewed as data defined on regular grids with uniform constant density. In CNNs, the counterpart to local partition scale is the size of kernels. [25] shows that using smaller kernels helps to improve the ability of CNNs. Our experiments on point set data, however, give counter evidence to this rule. Small neighborhood may consist of too few points due to sampling deficiency, which might be insufficient to allow PointNets to capture patterns robustly.

A significant contribution of our paper is that PointNet++ leverages neighborhoods at multiple scales to achieve both robustness and detail capture. Assisted with random input dropout during training, the network learns to adaptively weight patterns detected at different scales and combine multi-scale features according to the input data. Experiments show that our PointNet++ is able to process point sets efficiently and robustly. In particular, results that are significantly better than state-of-the-art have been obtained on challenging benchmarks of 3D point clouds.

2 Problem Statement

Suppose that $\mathcal{X} = (M, d)$ is a discrete metric space whose metric is inherited from a Euclidean space $\mathbb{R}^n$, where $M \subseteq \mathbb{R}^n$ is the set of points and d is the distance metric. In addition, the density of M in the ambient Euclidean space may not be uniform everywhere. We are interested in learning set functions f that take such $\mathcal{X}$ as the input (along with additional features for each point) and produce information of semantic interest regrading $\mathcal{X}$. In practice, such f can be classification function that assigns a label to $\mathcal{X}$ or a segmentation function that assigns a per point label to each member of M.

3 Method

Our work can be viewed as an extension of PointNet [20] with added hierarchical structure. We first review PointNet (Sec. 3.1) and then introduce a basic extension of PointNet with hierarchical structure (Sec. 3.2). Finally, we propose our PointNet++ that is able to robustly learn features even in non-uniformly sampled point sets (Sec. 3.3).

3.1 Review of PointNet [20]: A Universal Continuous Set Function Approximator

Given an unordered point set $\{x_1, x_2, ..., x_n\}$ with $x_i \in \mathbb{R}^d$, one can define a set function $f : \mathcal{X} \to \mathbb{R}$ that maps a set of points to a vector:

$$f(x_1, x_2, ..., x_n) = \gamma \left(\underset{i=1,\dots,n}{\text{MAX}} \{h(x_i)\} \right) \tag{1}$$

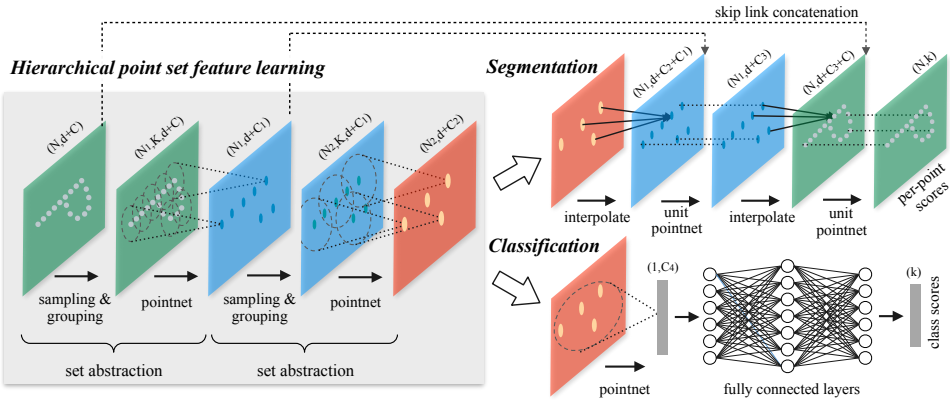


Figure 2: Illustration of our hierarchical feature learning architecture and its application for set segmentation and classification using points in 2D Euclidean space as an example. Single scale point grouping is visualized here. For details on density adaptive grouping, see Fig. 3

where γ and h are usually multi-layer perceptron (MLP) networks.

The set function f in Eq. 1 is invariant to input point permutations and can arbitrarily approximate any continuous set function [20]. Note that the response of h can be interpreted as the spatial encoding of a point (see [20] for details).

PointNet achieved impressive performance on a few benchmarks. However, it lacks the ability to capture local context at different scales. We will introduce a hierarchical feature learning framework in the next section to resolve the limitation.

3.2 Hierarchical Point Set Feature Learning

While PointNet uses a single max pooling operation to aggregate the whole point set, our new architecture builds a hierarchical grouping of points and progressively abstract larger and larger local regions along the hierarchy.

Our hierarchical structure is composed by a number of *set abstraction* levels (Fig. 2). At each level, a set of points is processed and abstracted to produce a new set with fewer elements. The set abstraction level is made of three key layers: *Sampling layer*, *Grouping layer* and *PointNet layer*. The *Sampling layer* selects a set of points from input points, which defines the centroids of local regions. *Grouping layer* then constructs local region sets by finding "neighboring" points around the centroids. *PointNet layer* uses a mini-PointNet to encode local region patterns into feature vectors.

A set abstraction level takes an $N \times (d + C)$ matrix as input that is from N points with d-dim coordinates and C-dim point feature. It outputs an $N' \times (d + C')$ matrix of N' subsampled points with d-dim coordinates and new C'-dim feature vectors summarizing local context. We introduce the layers of a set abstraction level in the following paragraphs.

Sampling layer. Given input points $\{x_1, x_2, ..., x_n\}$, we use iterative farthest point sampling (FPS) to choose a subset of points $\{x_{i_1}, x_{i_2}, ..., x_{i_m}\}$, such that x_{i_j} is the most distant point (in metric distance) from the set $\{x_{i_1}, x_{i_2}, ..., x_{i_{j-1}}\}$ with regard to the rest points. Compared with random sampling, it has better coverage of the entire point set given the same number of centroids. In contrast to CNNs that scan the vector space agnostic of data distribution, our sampling strategy generates receptive fields in a data dependent manner.

Grouping layer. The input to this layer is a point set of size $N \times (d + C)$ and the coordinates of a set of centroids of size $N' \times d$. The output are groups of point sets of size $N' \times K \times (d + C)$, where each group corresponds to a local region and K is the number of points in the neighborhood of centroid points. Note that K varies across groups but the succeeding *PointNet layer* is able to convert flexible number of points into a fixed length local region feature vector.

In convolutional neural networks, a local region of a pixel consists of pixels with array indices within certain Manhattan distance (kernel size) of the pixel. In a point set sampled from a metric space, the neighborhood of a point is defined by metric distance.

Ball query finds all points that are within a radius to the query point (an upper limit of K is set in implementation). An alternative range query is K nearest neighbor (kNN) search which finds a fixed

number of neighboring points. Compared with kNN, ball query's local neighborhood guarantees a fixed region scale thus making local region feature more generalizable across space, which is preferred for tasks requiring local pattern recognition (e.g. semantic point labeling).

PointNet layer. In this layer, the input are N' local regions of points with data size $N' \times K \times (d+C)$. Each local region in the output is abstracted by its centroid and local feature that encodes the centroid's neighborhood. Output data size is $N' \times (d + C')$.

The coordinates of points in a local region are firstly translated into a local frame relative to the centroid point: $x_i^{(j)} = x_i^{(j)} - \hat{x}^{(j)}$ for $i = 1, 2, ..., K$ and $j = 1, 2, ..., d$ where $\hat{x}$ is the coordinate of the centroid. We use PointNet [20] as described in Sec. 3.1 as the basic building block for local pattern learning. By using relative coordinates together with point features we can capture point-to-point relations in the local region.

3.3 Robust Feature Learning under Non-Uniform Sampling Density

As discussed earlier, it is common that a point set comes with non-uniform density in different areas. Such non-uniformity introduces a significant challenge for point set feature learning. Features learned in dense data may not generalize to sparsely sampled regions. Consequently, models trained for sparse point cloud may not recognize fine-grained local structures.

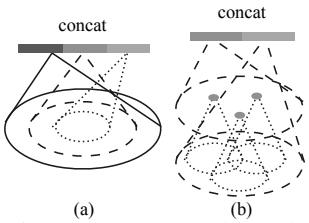


Figure 3: (a) Multi-scale grouping (MSG); (b) Multi-resolution grouping (MRG).

Ideally, we want to inspect as closely as possible into a point set to capture finest details in densely sampled regions. However, such close inspect is prohibited at low density areas because local patterns may be corrupted by the sampling deficiency. In this case, we should look for larger scale patterns in greater vicinity. To achieve this goal we propose density adaptive PointNet layers (Fig. 3) that learn to combine features from regions of different scales when the input sampling density changes. We call our hierarchical network with density adaptive PointNet layers as *PointNet++*.

Previously in Sec. 3.2, each abstraction level contains grouping and feature extraction of a single scale. In PointNet++, each abstraction level extracts multiple scales of local patterns and combine them intelligently according to local point densities. In terms of grouping local regions and combining features from different scales, we propose two types of density adaptive layers as listed below.

Multi-scale grouping (MSG). As shown in Fig. 3 (a), a simple but effective way to capture multi-scale patterns is to apply grouping layers with different scales followed by according PointNets to extract features of each scale. Features at different scales are concatenated to form a multi-scale feature.

We train the network to learn an optimized strategy to combine the multi-scale features. This is done by randomly dropping out input points with a randomized probability for each instance, which we call *random input dropout*. Specifically, for each training point set, we choose a dropout ratio θ uniformly sampled from $[0, p]$ where $p \leq 1$. For each point, we randomly drop a point with probability θ. In practice we set $p = 0.95$ to avoid generating empty point sets. In doing so we present the network with training sets of various sparsity (induced by θ) and varying uniformity (induced by randomness in dropout). During test, we keep all available points.

Multi-resolution grouping (MRG). The MSG approach above is computationally expensive since it runs local PointNet at large scale neighborhoods for every centroid point. In particular, since the number of centroid points is usually quite large at the lowest level, the time cost is significant.

Here we propose an alternative approach that avoids such expensive computation but still preserves the ability to adaptively aggregate information according to the distributional properties of points. In Fig. 3 (b), features of a region at some level L_i is a concatenation of two vectors. One vector (left in figure) is obtained by summarizing the features at each subregion from the lower level L_{i-1} using the set abstraction level. The other vector (right) is the feature that is obtained by directly processing all raw points in the local region using a single PointNet.

When the density of a local region is low, the first vector may be less reliable than the second vector, since the subregion in computing the first vector contains even sparser points and suffers more from sampling deficiency. In such a case, the second vector should be weighted higher. On the other hand,

when the density of a local region is high, the first vector provides information of finer details since it possesses the ability to inspect at higher resolutions recursively in lower levels.

Compared with MSG, this method is computationally more efficient since we avoids the feature extraction in large scale neighborhoods at lowest levels.

3.4 Point Feature Propagation for Set Segmentation

In set abstraction layer, the original point set is subsampled. However in set segmentation task such as semantic point labeling, we want to obtain point features for *all* the original points. One solution is to always sample all points as centroids in all set abstraction levels, which however results in high computation cost. Another way is to propagate features from subsampled points to the original points.

We adopt a hierarchical propagation strategy with distance based interpolation and across level skip links (as shown in Fig. 2). In a *feature propagation* level, we propagate point features from $N_l \times (d + C)$ points to N_{l-1} points where N_{l-1} and N_l (with $N_l \leq N_{l-1}$) are point set size of input and output of set abstraction level l. We achieve feature propagation by interpolating feature values f of N_l points at coordinates of the N_{l-1} points. Among the many choices for interpolation, we use inverse distance weighted average based on k nearest neighbors (as in Eq. 2, in default we use $p = 2$, $k = 3$). The interpolated features on N_{l-1} points are then concatenated with skip linked point features from the set abstraction level. Then the concatenated features are passed through a "unit pointnet", which is similar to one-by-one convolution in CNNs. A few shared fully connected and ReLU layers are applied to update each point's feature vector. The process is repeated until we have propagated features to the original set of points.

$$f^{(j)}(x) = \frac{\sum_{i=1}^{k} w_i(x) f_i^{(j)}}{\sum_{i=1}^{k} w_i(x)} \quad \text{where} \quad w_i(x) = \frac{1}{d(x, x_i)^p},\ j = 1, ..., C \tag{2}$$

4 Experiments

Datasets We evaluate on four datasets ranging from 2D objects (MNIST [11]), 3D objects (ModelNet40 [31] rigid object, SHREC15 [12] non-rigid object) to real 3D scenes (ScanNet [5]). Object classification is evaluated by accuracy. Semantic scene labeling is evaluated by average voxel classification accuracy following [5]. We list below the experiment setting for each dataset:

- MNIST: Images of handwritten digits with 60k training and 10k testing samples.
- ModelNet40: CAD models of 40 categories (mostly man-made). We use the official split with 9,843 shapes for training and 2,468 for testing.
- SHREC15: 1200 shapes from 50 categories. Each category contains 24 shapes which are mostly organic ones with various poses such as horses, cats, etc. We use five fold cross validation to acquire classification accuracy on this dataset.
- ScanNet: 1513 scanned and reconstructed indoor scenes. We follow the experiment setting in [5] and use 1201 scenes for training, 312 scenes for test.

4.1 Point Set Classification in Euclidean Metric Space

We evaluate our network on classifying point clouds sampled from both 2D (MNIST) and 3D (ModleNet40) Euclidean spaces. MNIST images are converted to 2D point clouds of digit pixel locations. 3D point clouds are sampled from mesh surfaces from ModelNet40 shapes. In default we use 512 points for MNIST and 1024 points for ModelNet40. In last row (ours normal) in Table 2, we use face normals as additional point features, where we also use more points ($N = 5000$) to further boost performance. All point sets are normalized to be zero mean and within a unit ball. We use a three-level hierarchical network with three fully connected layers [1]

Results. In Table 1 and Table 2, we compare our method with a representative set of previous state of the arts. Note that PointNet (vanilla) in Table 2 is the the version in [20] that does not use transformation networks, which is equivalent to our hierarchical net with only one level.

Firstly, our hierarchical learning architecture achieves significantly better performance than the non-hierarchical PointNet [20]. In MNIST, we see a relative 60.8% and 34.6% error rate reduction

[1] See supplementary for more details on network architecture and experiment preparation.

Method	Error rate (%)
Multi-layer perceptron [24]	1.60
LeNet5 [11]	0.80
Network in Network [13]	**0.47**
PointNet (vanilla) [20]	1.30
PointNet [20]	0.78
Ours	0.51

Table 1: MNIST digit classification.

Method	Input	Accuracy (%)
Subvolume [21]	vox	89.2
MVCNN [26]	img	90.1
PointNet (vanilla) [20]	pc	87.2
PointNet [20]	pc	89.2
Ours	pc	90.7
Ours (with normal)	pc	**91.9**

Table 2: ModelNet40 shape classification.

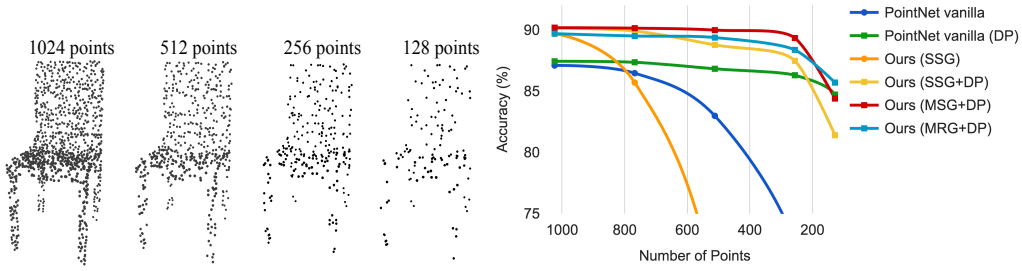


Figure 4: Left: Point cloud with random point dropout. Right: Curve showing advantage of our density adaptive strategy in dealing with non-uniform density. DP means random input dropout during training; otherwise training is on uniformly dense points. See Sec.3.3 for details.

from PointNet (vanilla) and PointNet to our method. In ModelNet40 classification, we also see that using same input data size (1024 points) and features (coordinates only), ours is remarkably stronger than PointNet. Secondly, we observe that point set based method can even achieve better or similar performance as mature image CNNs. In MNIST, our method (based on 2D point set) is achieving an accuracy close to the Network in Network CNN. In ModelNet40, ours with normal information significantly outperforms previous state-of-the-art method MVCNN [26].

Robustness to Sampling Density Variation. Sensor data directly captured from real world usually suffers from severe irregular sampling issues (Fig. 1). Our approach selects point neighborhood of multiple scales and learns to balance the descriptiveness and robustness by properly weighting them.

We randomly drop points (see Fig. 4 left) during test time to validate our network's robustness to non-uniform and sparse data. In Fig. 4 right, we see MSG+DP (multi-scale grouping with random input dropout during training) and MRG+DP (multi-resolution grouping with random input dropout during training) are very robust to sampling density variation. MSG+DP performance drops by less than 1% from 1024 to 256 test points. Moreover, it achieves the best performance on almost all sampling densities compared with alternatives. PointNet vanilla [20] is fairly robust under density variation due to its focus on global abstraction rather than fine details. However loss of details also makes it less powerful compared to our approach. SSG (ablated PointNet++ with single scale grouping in each level) fails to generalize to sparse sampling density while SSG+DP amends the problem by randomly dropping out points in training time.

4.2 Point Set Segmentation for Semantic Scene Labeling

To validate that our approach is suitable for large scale point cloud analysis, we also evaluate on semantic scene labeling task. The goal is to predict semantic object label for points in indoor scans. [5] provides a baseline using fully convolutional neural network on voxelized scans. They purely rely on scanning geometry instead of RGB information and report the accuracy on a per-voxel basis. To make a fair comparison, we remove RGB information in all our experiments and convert point cloud label prediction into voxel labeling following [5]. We also compare with [20]. The accuracy is reported on a per-voxel basis in Fig. 5 (blue bar).

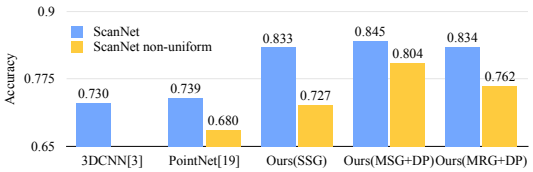


Figure 5: Scannet labeling accuracy.

Our approach outperforms all the baseline methods by a large margin. In comparison with [5], which learns on voxelized scans, we directly learn on point clouds to avoid additional quantization error,

and conduct data dependent sampling to allow more effective learning. Compared with [20], our approach introduces hierarchical feature learning and captures geometry features at different scales. This is very important for understanding scenes at multiple levels and labeling objects with various sizes. We visualize example scene labeling results in Fig. 6.

Robustness to Sampling Density Variation To test how our trained model performs on scans with non-uniform sampling density, we synthesize virtual scans of Scannet scenes similar to that in Fig. 1 and evaluate our network on this data. We refer readers to supplementary material for how we generate the virtual scans. We evaluate our framework in three settings (SSG, MSG+DP, MRG+DP) and compare with a baseline approach [20].

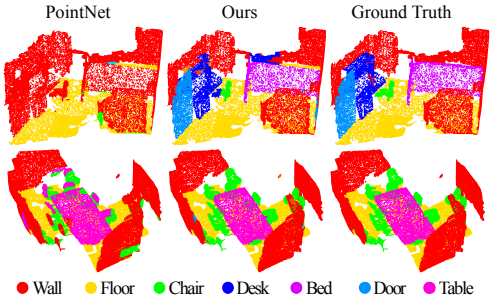


Figure 6: Scannet labeling results. [20] captures the overall layout of the room correctly but fails to discover the furniture. Our approach, in contrast, is much better at segmenting objects besides the room layout.

Performance comparison is shown in Fig. 5 (yellow bar). We see that SSG performance greatly falls due to the sampling density shift from uniform point cloud to virtually scanned scenes. MRG network, on the other hand, is more robust to the sampling density shift since it is able to automatically switch to features depicting coarser granularity when the sampling is sparse. Even though there is a domain gap between training data (uniform points with random dropout) and scanned data with non-uniform density, our MSG network is only slightly affected and achieves the best accuracy among methods in comparison. These prove the effectiveness of our density adaptive layer design.

4.3 Point Set Classification in Non-Euclidean Metric Space

In this section, we show generalizability of our approach to non-Euclidean space. In non-rigid shape classification (Fig. 7), a good classifier should be able to classify (a) and (c) in Fig. 7 correctly as the same category even given their difference in pose, which requires knowledge of intrinsic structure. Shapes in SHREC15 are 2D surfaces embedded in 3D space. Geodesic distances along the surfaces naturally induce a metric space. We show through experiments that adopting PointNet++ in this metric space is an effective way to capture intrinsic structure of the underlying point set.

For each shape in [12], we firstly construct the metric space induced by pairwise geodesic distances. We follow [23] to obtain an embedding metric that mimics geodesic distance. Next we extract intrinsic point features in this metric space including WKS [1], HKS [27] and multi-scale Gaussian curvature [16]. We use these features as input and then sample and group points according to the underlying metric space. In this way, our network learns to capture multi-scale intrinsic structure that is not influenced by the specific pose of a shape. Alternative design choices include using XYZ coordinates as points feature or use Euclidean space $\mathbb{R}^3$ as the underlying metric space. We show below these are not optimal choices.

(a) Horse (b) Cat (c) Horse
Figure 7: An example of non-rigid shape classification.

Results. We compare our methods with previous state-of-the-art method [14] in Table 3. [14] extracts geodesic moments as shape features and use a stacked sparse autoencoder to digest these features to predict shape category. Our approach using non-Euclidean metric space and intrinsic features achieves the best performance in all settings and outperforms [14] by a large margin.

Comparing the first and second setting of our approach, we see intrinsic features are very important for non-rigid shape classification. XYZ feature fails to reveal intrinsic structures and is greatly influenced by pose variation. Comparing the second and third setting of our approach, we see using geodesic neighborhood is beneficial compared with Euclidean neighborhood. Euclidean neighborhood might include points far away on surfaces and this neighborhood could change dramatically when shape affords non-rigid deformation. This introduces difficulty for effective weight sharing since the local structure could become combinatorially complicated. Geodesic neighborhood on surfaces, on the other hand, gets rid of this issue and improves the learning effectiveness.

	Metric space	Input feature	Accuracy (%)
DeepGM [14]	-	Intrinsic features	93.03
Ours	Euclidean	XYZ	60.18
	Euclidean	Intrinsic features	94.49
	Non-Euclidean	Intrinsic features	**96.09**

Table 3: SHREC15 Non-rigid shape classification.

4.4 Feature Visualization.

In Fig. 8 we visualize what has been learned by the first level kernels of our hierarchical network. We created a voxel grid in space and aggregate local point sets that activate certain neurons the most in grid cells (highest 100 examples are used). Grid cells with high votes are kept and converted back to 3D point clouds, which represents the pattern that neuron recognizes. Since the model is trained on ModelNet40 which is mostly consisted of furniture, we see structures of planes, double planes, lines, corners etc. in the visualization.

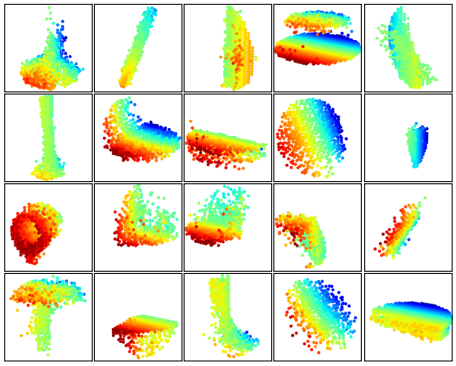


Figure 8: 3D point cloud patterns learned from the first layer kernels. The model is trained for ModelNet40 shape classification (20 out of the 128 kernels are randomly selected). Color indicates point depth (red is near, blue is far).

5 Related Work

The idea of hierarchical feature learning has been very successful. Among all the learning models, convolutional neural network [10, 25, 8] is one of the most prominent ones. However, convolution does not apply to unordered point sets with distance metrics, which is the focus of our work.

A few very recent works [20, 28] have studied how to apply deep learning to unordered sets. They ignore the underlying distance metric even if the point set does possess one. As a result, they are unable to capture local context of points and are sensitive to global set translation and normalization. In this work, we target at points sampled from a metric space and tackle these issues by explicitly considering the underlying distance metric in our design.

Point sampled from a metric space are usually noisy and with non-uniform sampling density. This affects effective point feature extraction and causes difficulty for learning. One of the key issue is to select proper scale for point feature design. Previously several approaches have been developed regarding this [19, 17, 2, 6, 7, 30] either in geometry processing community or photogrammetry and remote sensing community. In contrast to all these works, our approach learns to extract point features and balance multiple feature scales in an end-to-end fashion.

In 3D metric space, other than point set, there are several popular representations for deep learning, including volumetric grids [21, 22, 29], and geometric graphs [3, 15, 33]. However, in none of these works, the problem of non-uniform sampling density has been explicitly considered.

6 Conclusion

In this work, we propose PointNet++, a powerful neural network architecture for processing point sets sampled in a metric space. PointNet++ recursively functions on a nested partitioning of the input point set, and is effective in learning hierarchical features with respect to the distance metric. To handle the non uniform point sampling issue, we propose two novel set abstraction layers that intelligently aggregate multi-scale information according to local point densities. These contributions enable us to achieve state-of-the-art performance on challenging benchmarks of 3D point clouds.

In the future, it's worthwhile thinking how to accelerate inference speed of our proposed network especially for MSG and MRG layers by sharing more computation in each local regions. It's also interesting to find applications in higher dimensional metric spaces where CNN based method would be computationally unfeasible while our method can scale well.

References

[1] M. Aubry, U. Schlickewei, and D. Cremers. The wave kernel signature: A quantum mechanical approach to shape analysis. In *Computer Vision Workshops (ICCV Workshops), 2011 IEEE International Conference on*, pages 1626–1633. IEEE, 2011.

[2] D. Belton and D. D. Lichti. Classification and segmentation of terrestrial laser scanner point clouds using local variance information. *Iaprs, Xxxvi*, 5:44–49, 2006.

[3] J. Bruna, W. Zaremba, A. Szlam, and Y. LeCun. Spectral networks and locally connected networks on graphs. *arXiv preprint arXiv:1312.6203*, 2013.

[4] A. X. Chang, T. Funkhouser, L. Guibas, P. Hanrahan, Q. Huang, Z. Li, S. Savarese, M. Savva, S. Song, H. Su, J. Xiao, L. Yi, and F. Yu. ShapeNet: An Information-Rich 3D Model Repository. Technical Report arXiv:1512.03012 [cs.GR], 2015.

[5] A. Dai, A. X. Chang, M. Savva, M. Halber, T. Funkhouser, and M. Nießner. Scannet: Richly-annotated 3d reconstructions of indoor scenes. *arXiv preprint arXiv:1702.04405*, 2017.

[6] J. Demantké, C. Mallet, N. David, and B. Vallet. Dimensionality based scale selection in 3d lidar point clouds. *The International Archives of the Photogrammetry, Remote Sensing and Spatial Information Sciences*, 38(Part 5):W12, 2011.

[7] A. Gressin, C. Mallet, J. Demantké, and N. David. Towards 3d lidar point cloud registration improvement using optimal neighborhood knowledge. *ISPRS journal of photogrammetry and remote sensing*, 79:240–251, 2013.

[8] K. He, X. Zhang, S. Ren, and J. Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 770–778, 2016.

[9] D. Kingma and J. Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*.

[10] A. Krizhevsky, I. Sutskever, and G. E. Hinton. Imagenet classification with deep convolutional neural networks. In *Advances in neural information processing systems*, pages 1097–1105, 2012.

[11] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.

[12] Z. Lian, J. Zhang, S. Choi, H. ElNaghy, J. El-Sana, T. Furuya, A. Giachetti, R. A. Guler, L. Lai, C. Li, H. Li, F. A. Limberger, R. Martin, R. U. Nakanishi, A. P. Neto, L. G. Nonato, R. Ohbuchi, K. Pevzner, D. Pickup, P. Rosin, A. Sharf, L. Sun, X. Sun, S. Tari, G. Unal, and R. C. Wilson. Non-rigid 3D Shape Retrieval. In I. Pratikakis, M. Spagnuolo, T. Theoharis, L. V. Gool, and R. Veltkamp, editors, *Eurographics Workshop on 3D Object Retrieval*. The Eurographics Association, 2015.

[13] M. Lin, Q. Chen, and S. Yan. Network in network. *arXiv preprint arXiv:1312.4400*, 2013.

[14] L. Luciano and A. B. Hamza. Deep learning with geodesic moments for 3d shape classification. *Pattern Recognition Letters*, 2017.

[15] J. Masci, D. Boscaini, M. Bronstein, and P. Vandergheynst. Geodesic convolutional neural networks on riemannian manifolds. In *Proceedings of the IEEE International Conference on Computer Vision Workshops*, pages 37–45, 2015.

[16] M. Meyer, M. Desbrun, P. Schröder, A. H. Barr, et al. Discrete differential-geometry operators for triangulated 2-manifolds. *Visualization and mathematics*, 3(2):52–58, 2002.

[17] N. J. MITRA, A. NGUYEN, and L. GUIBAS. Estimating surface normals in noisy point cloud data. *International Journal of Computational Geometry & Applications*, 14(04n05):261–276, 2004.

[18] I. Occipital. Structure sensor-3d scanning, augmented reality, and more for mobile devices, 2016.

[19] M. Pauly, L. P. Kobbelt, and M. Gross. Point-based multiscale surface representation. *ACM Transactions on Graphics (TOG)*, 25(2):177–193, 2006.

[20] C. R. Qi, H. Su, K. Mo, and L. J. Guibas. Pointnet: Deep learning on point sets for 3d classification and segmentation. *arXiv preprint arXiv:1612.00593*, 2016.

[21] C. R. Qi, H. Su, M. Nießner, A. Dai, M. Yan, and L. Guibas. Volumetric and multi-view cnns for object classification on 3d data. In *Proc. Computer Vision and Pattern Recognition (CVPR), IEEE*, 2016.

[22] G. Riegler, A. O. Ulusoys, and A. Geiger. Octnet: Learning deep 3d representations at high resolutions. *arXiv preprint arXiv:1611.05009*, 2016.

[23] R. M. Rustamov, Y. Lipman, and T. Funkhouser. Interior distance using barycentric coordinates. In *Computer Graphics Forum*, volume 28, pages 1279–1288. Wiley Online Library, 2009.

[24] P. Y. Simard, D. Steinkraus, and J. C. Platt. Best practices for convolutional neural networks applied to visual document analysis. In *ICDAR*, volume 3, pages 958–962, 2003.

[25] K. Simonyan and A. Zisserman. Very deep convolutional networks for large-scale image recognition. *arXiv preprint arXiv:1409.1556*, 2014.

[26] H. Su, S. Maji, E. Kalogerakis, and E. G. Learned-Miller. Multi-view convolutional neural networks for 3d shape recognition. In *Proc. ICCV, to appear*, 2015.

[27] J. Sun, M. Ovsjanikov, and L. Guibas. A concise and provably informative multi-scale signature based on heat diffusion. In *Computer graphics forum*, volume 28, pages 1383–1392. Wiley Online Library, 2009.

[28] O. Vinyals, S. Bengio, and M. Kudlur. Order matters: Sequence to sequence for sets. *arXiv preprint arXiv:1511.06391*, 2015.

[29] P.-S. WANG, Y. LIU, Y.-X. GUO, C.-Y. SUN, and X. TONG. O-cnn: Octree-based convolutional neural networks for 3d shape analysis. 2017.

[30] M. Weinmann, B. Jutzi, S. Hinz, and C. Mallet. Semantic point cloud interpretation based on optimal neighborhoods, relevant features and efficient classifiers. *ISPRS Journal of Photogrammetry and Remote Sensing*, 105:286–304, 2015.

[31] Z. Wu, S. Song, A. Khosla, F. Yu, L. Zhang, X. Tang, and J. Xiao. 3d shapenets: A deep representation for volumetric shapes. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 1912–1920, 2015.

[32] L. Yi, V. G. Kim, D. Ceylan, I.-C. Shen, M. Yan, H. Su, C. Lu, Q. Huang, A. Sheffer, and L. Guibas. A scalable active framework for region annotation in 3d shape collections. *SIGGRAPH Asia*, 2016.

[33] L. Yi, H. Su, X. Guo, and L. Guibas. Syncspeccnn: Synchronized spectral cnn for 3d shape segmentation. *arXiv preprint arXiv:1612.00606*, 2016.

Supplementary

A Overview

This supplementary material provides more details on experiments in the main paper and includes more experiments to validate and analyze our proposed method.

In Sec B we provide specific network architectures used for experiments in the main paper and also describe details in data preparation and training. In Sec C we show more experimental results including benchmark performance on part segmentation and analysis on neighborhood query, sensitivity to sampling randomness and time space complexity.

B Details in Experiments

Architecture protocol. We use following notations to describe our network architecture.

SA(K,r,[$l_1, ..., l_d$]) is a set abstraction (SA) level with K local regions of ball radius r using PointNet of d fully connected layers with width l_i ($i = 1, ..., d$). SA([$l_1, ...l_d$]) is a global set abstraction level that converts set to a single vector. In multi-scale setting (as in MSG), we use SA(K, [$r^{(1)}, ..., r^{(m)}$], [[$l_1^{(1)}, ..., l_d^{(1)}$],...,[$l_1^{(m)}, ..., l_d^{(m)}$]]) to represent MSG with m scales.

FC(l,dp) represents a fully connected layer with width l and dropout ratio dp. FP($l_1, ..., l_d$) is a feature propagation (FP) level with d fully connected layers. It is used for updating features concatenated from interpolation and skip link. All fully connected layers are followed by batch normalization and ReLU except for the last score prediction layer.

B.1 Network Architectures

For all classification experiments we use the following architecture (Ours SSG) with different K (number of categories):

$$SA(512, 0.2, [64, 64, 128]) \rightarrow SA(128, 0.4, [128, 128, 256]) \rightarrow SA([256, 512, 1024]) \rightarrow FC(512, 0.5) \rightarrow FC(256, 0.5) \rightarrow FC(K)$$

The multi-scale grouping (MSG) network (PointNet++) architecture is as follows:

$$SA(512, [0.1, 0.2, 0.4], [[32, 32, 64], [64, 64, 128], [64, 96, 128]]) \rightarrow SA(128, [0.2, 0.4, 0.8], [[64, 64, 128], [128, 128, 256], [128, 128, 256]]) \rightarrow SA([256, 512, 1024]) \rightarrow FC(512, 0.5) \rightarrow FC(256, 0.5) \rightarrow FC(K)$$

The cross level multi-resolution grouping (MRG) network's architecture uses three branches:

Branch 1: $SA(512, 0.2, [64, 64, 128]) \rightarrow SA(64, 0.4, [128, 128, 256])$

Branch 2: $SA(512, 0.4, [64, 128, 256])$ using $r = 0.4$ regions of original points

Branch 3: $SA(64, 128, 256, 512)$ using all original points.

Branch 4: $SA(256, 512, 1024)$.

Branch 1 and branch 2 are concatenated and fed to branch 4. Output of branch 3 and branch4 are then concatenated and fed to $FC(512, 0.5) \rightarrow FC(256, 0.5) \rightarrow FC(K)$ for classification.

Network for semantic scene labeling (last two fully connected layers in FP are followed by dropout layers with drop ratio 0.5):

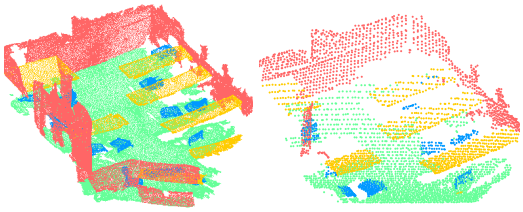
$$SA(1024, 0.1, [32, 32, 64]) \rightarrow SA(256, 0.2, [64, 64, 128]) \rightarrow SA(64, 0.4, [128, 128, 256]) \rightarrow SA(16, 0.8, [256, 256, 512]) \rightarrow FP(256, 256) \rightarrow FP(256, 256) \rightarrow FP(256, 128) \rightarrow FP(128, 128, 128, 128, K)$$

Network for semantic and part segmentation (last two fully connected layers in FP are followed by dropout layers with drop ratio 0.5):

$$SA(512, 0.2, [64, 64, 128]) \rightarrow SA(128, 0.4, [128, 128, 256]) \rightarrow SA([256, 512, 1024]) \rightarrow FP(256, 256) \rightarrow FP(256, 128) \rightarrow FP(128, 128, 128, 128, K)$$

B.2 Virtual Scan Generation

In this section, we describe how we generate labeled virtual scan with non-uniform sampling density from ScanNet scenes. For each scene in ScanNet, we set camera location $1.5m$ above the centroid of the floor plane and rotate the camera orientation in the horizontal plane evenly in 8 directions. In each direction, we use a image plane with size $100px$ by $75px$ and cast rays from camera through each pixel to the scene. This gives a way to select visible points in the scene. We could then generate 8 virtual scans for each test scene similar and an example is shown in Fig. 9. Notice point samples are denser in regions closer to the camera.



(a) ScanNet labeled scene (b) ScanNet non-uniform

Figure 9: Virtual scan generated from ScanNet

B.3 MNIST and ModelNet40 Experiment Details

For MNIST images, we firstly normalize all pixel intensities to range $[0, 1]$ and then select all pixels with intensities larger than 0.5 as valid digit pixels. Then we convert digit pixels in an image into a 2D point cloud with coordinates within $[-1, 1]$, where the image center is the origin point. Augmented points are created to add the point set up to a fixed cardinality (512 in our case). We jitter the initial point cloud (with random translation of Gaussian distribution $\mathcal{N}(0, 0.01)$ and clipped to 0.03) to generate the augmented points. For ModelNet40, we uniformly sample N points from CAD models surfaces based on face area.

For all experiments, we use Adam [9] optimizer with learning rate 0.001 for training. For data augmentation, we randomly scale object, perturb the object location as well as point sample locations. We also follow [21] to randomly rotate objects for ModelNet40 data augmentation. We use TensorFlow and GTX 1080, Titan X for training. All layers are implemented in CUDA to run GPU. It takes around 20 hours to train our model to convergence.

B.4 ScanNet Experiment Details

To generate training data from ScanNet scenes, we sample 1.5m by 1.5m by 3m cubes from the initial scene and then keep the cubes where $\geq 2\%$ of the voxels are occupied and $\geq 70\%$ of the surface voxels have valid annotations (this is the same set up in [5]). We sample such training cubes on the fly and random rotate it along the up-right axis. Augmented points are added to the point set to make a fixed cardinality (8192 in our case). During test time, we similarly split the test scene into smaller cubes and get label prediction for every point in the cubes first, then merge label prediction in all the cubes from a same scene. If a point get different labels from different cubes, we will just conduct a majority voting to get the final point label prediction.

B.5 SHREC15 Experiment Details

We randomly sample 1024 points on each shape both for training and testing. To generate the input intrinsic features, we to extract 100 dimensional WKS, HKS and multiscale Gaussian curvature respectively, leading to a 300 dimensional feature vector for each point. Then we conduct PCA to reduce the feature dimension to 64. We use a 8 dimensional embedding following [23] to mimic the geodesic distance, which is used to describe our non-Euclidean metric space while choosing the point neighborhood.

C More Experiments

In this section we provide more experiment results to validate and analyze our proposed network architecture.

C.1 Semantic Part Segmentation

Following the setting in [32], we evaluate our approach on part segmentation task assuming category label for each shape is already known. Taken shapes represented by point clouds as input, the task is to predict a part label for each point. The dataset contains 16,881 shapes from 16 classes, annotated with 50 parts in total. We use the official train test split following [4].

We equip each point with its normal direction to better depict the underlying shape. This way we could get rid of hand-crafted geometric features as is used in [32, 33]. We compare our framework with traditional learning based techniques [32], as well as state-of-the-art deep learning approaches [20, 33] in Table 4. Point intersection over union (IoU) is used as the evaluation metric, averaged across all part classes. Cross-entropy loss is minimized during training. On average, our approach achieves the best performance. In comparison with [20], our approach performs better on most of the categories, which proves the importance of hierarchical feature learning for detailed semantic understanding. Notice our approach could be viewed as implicitly building proximity graphs at different scales and operating on these graphs, thus is related to graph CNN approaches such as [33]. Thanks to the flexibility of our multi-scale neighborhood selection as well as the power of set operation units, we could achieve better performance compared with [33]. Notice our set operation unit is much simpler compared with graph convolution kernels, and we do not need to conduct expensive eigen decomposition as opposed to [33]. These make our approach more suitable for large scale point cloud analysis.

	mean	aero	bag	cap	car	chair	ear phone	guitar	knife	lamp	laptop	motor	mug	pistol	rocket	skate board	table
Yi [32]	81.4	81.0	78.4	77.7	75.7	87.6	61.9	92.0	85.4	82.5	95.7	70.6	91.9	85.9	53.1	69.8	75.3
PN [20]	83.7	83.4	78.7	82.5	74.9	89.6	73.0	91.5	85.9	80.8	95.3	65.2	93.0	81.2	57.9	72.8	80.6
SSCNN [33]	84.7	81.6	81.7	81.9	75.2	90.2	74.9	93.0	86.1	84.7	95.6	66.7	92.7	81.6	60.6	82.9	82.1
Ours	85.1	82.4	79.0	87.7	77.3	90.8	71.8	91.0	85.9	83.7	95.3	71.6	94.1	81.3	58.7	76.4	82.6

Table 4: Segmentation results on ShapeNet part dataset.

C.2 Neighborhood Query: kNN v.s. Ball Query.

Here we compare two options to select a local neighborhood. We used radius based ball query in our main paper. Here we also experiment with kNN based neighborhood search and also play with different search radius and k. In this experiment all training and testing are on ModelNet40 shapes with uniform sampling density. 1024 points are used. As seen in Table 5, radius based ball query is slightly better than kNN based method. However, we speculate in very non-uniform point set, kNN based query will results in worse generalization ability. Also we observe that a slightly large radius is helpful for performance probably because it captures richer local patterns.

kNN (k=16)	kNN (k=64)	radius (r=0.1)	radius (r=0.2)
89.3	90.3	89.1	90.7

Table 5: Effects of neighborhood choices. Evaluation metric is classification accuracy (%) on ModelNet 40 test set.

C.3 Effect of Randomness in Farthest Point Sampling.

For the *Sampling layer* in our set abstraction level, we use farthest point sampling (FPS) for point set sub sampling. However FPS algorithm is random and the subsampling depends on which point is selected first. Here we evaluate the sensitivity of our model to this randomness. In Table 6, we test our model trained on ModelNet40 for feature stability and classification stability.

To evaluate feature stability we extract global features of all test samples for 10 times with different random seed. Then we compute mean features for each shape across the 10 sampling. Then we compute standard deviation of the norms of feature's difference from the mean feature. At last we average all std. in all feature dimensions as reported in the table. Since features are normalized into 0 to 1 before processing, the 0.021 difference means a 2.1% deviation of feature norm.

For classification, we observe only a 0.17% standard deviation in test accuracy on all ModelNet40 test shapes, which is robust to sampling randomness.

Feature difference std.	Accuracy std.
0.021	0.0017

Table 6: Effects of randomness in FPS (using ModelNet40).

C.4 Time and Space Complexity.

Table 7 summarizes comparisons of time and space cost between a few point set based deep learning method. We record forward time with a batch size 8 using TensorFlow 1.1 with a single GTX 1080. The first batch is neglected since there is some preparation for GPU. While PointNet (vanilla) [20] has the best time efficiency, our model without density adaptive layers achieved smallest model size with fair speed.

It's worth noting that ours MSG, while it has good performance in non-uniformly sampled data, it's 2x expensive than SSG version due the multi-scale region feature extraction. Compared with MSG, MRG is more efficient since it uses regions across layers.

	PointNet (vanilla) 1	PointNet 1	Ours (SSG)	Ours (MSG)	Ours (MRG)
Model size (MB)	9.4	40	8.7	12	24
Forward time (ms)	11.6	25.3	82.4	163.2	87.0

Table 7: Model size and inference time (forward pass) of several networks.